Digital Signature를 이용한 무결성 기능을 지원하는

파일 전송 SW 구현

2026. 9. 7.

팀 해시브라운 

강정우
이원준
최재훈

INDEX

목차

1. 프로젝트 목표

2. 계획 대비 진행상황

3. 역할 분담

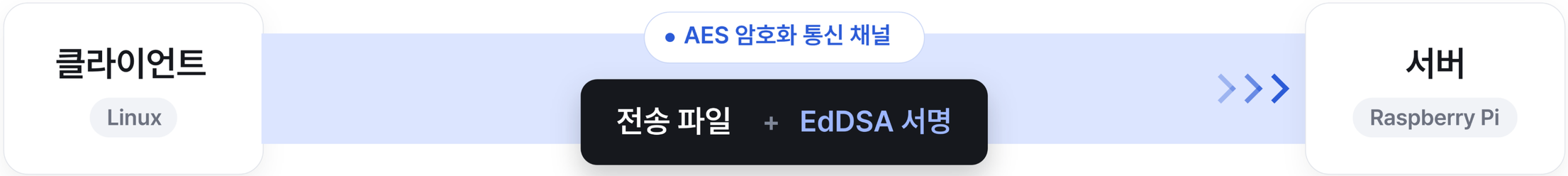
4. 설계 사양

5. 연구 내용 및 중간 결과물

6. 프로젝트 향후 계획



연구 목적



무결성 | EdDSA 디지털 서명

파일 해시에 서명하고 수신 측이 공개키로 검증하고,
검증에 실패하면 저장을 거부

기밀성 | AES 암호화

디지털 서명이 다루지 않는 내용 노출을 암호화 채널로 보완

안전한 파일 전송 소프트웨어 구현

연구 일정

연구내용	3월	4월	5월	6월	7월	8월	9월	10월
관련 내용 학습	■	■	■					
개발 환경 구축		■						
파일전송, 디지털 서명 모듈 구현		■	■	■				
AES 암호화 모듈 구현					■	■		
기능 개선							■	
라즈베리파이 동작 시험							■	
발표 자료 제작 및 시연 준비							■	■
최종 발표 및 산출물 제출								■

연구 일정

연구내용	3월	4월	5월	6월	7월	8월	9월	10월
관련 내용 학습	■	■	■					
개발 환경 구축		■						
파일전송, 디지털 서명 모듈 구현		■	■	■				
AES 암호화 모듈 구현					■	■		
기능 개선							■	
라즈베리파이 동작 시험							■	
발표 자료 제작 및 시연 준비							■	■
최종 발표 및 산출물 제출								■

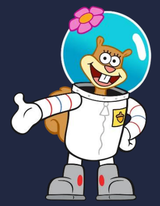
역할 분담

강정우



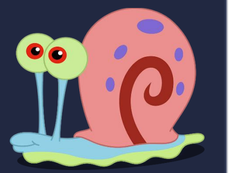
파일 전송 구현 및
OpenSSL 사용법 학습

이원준



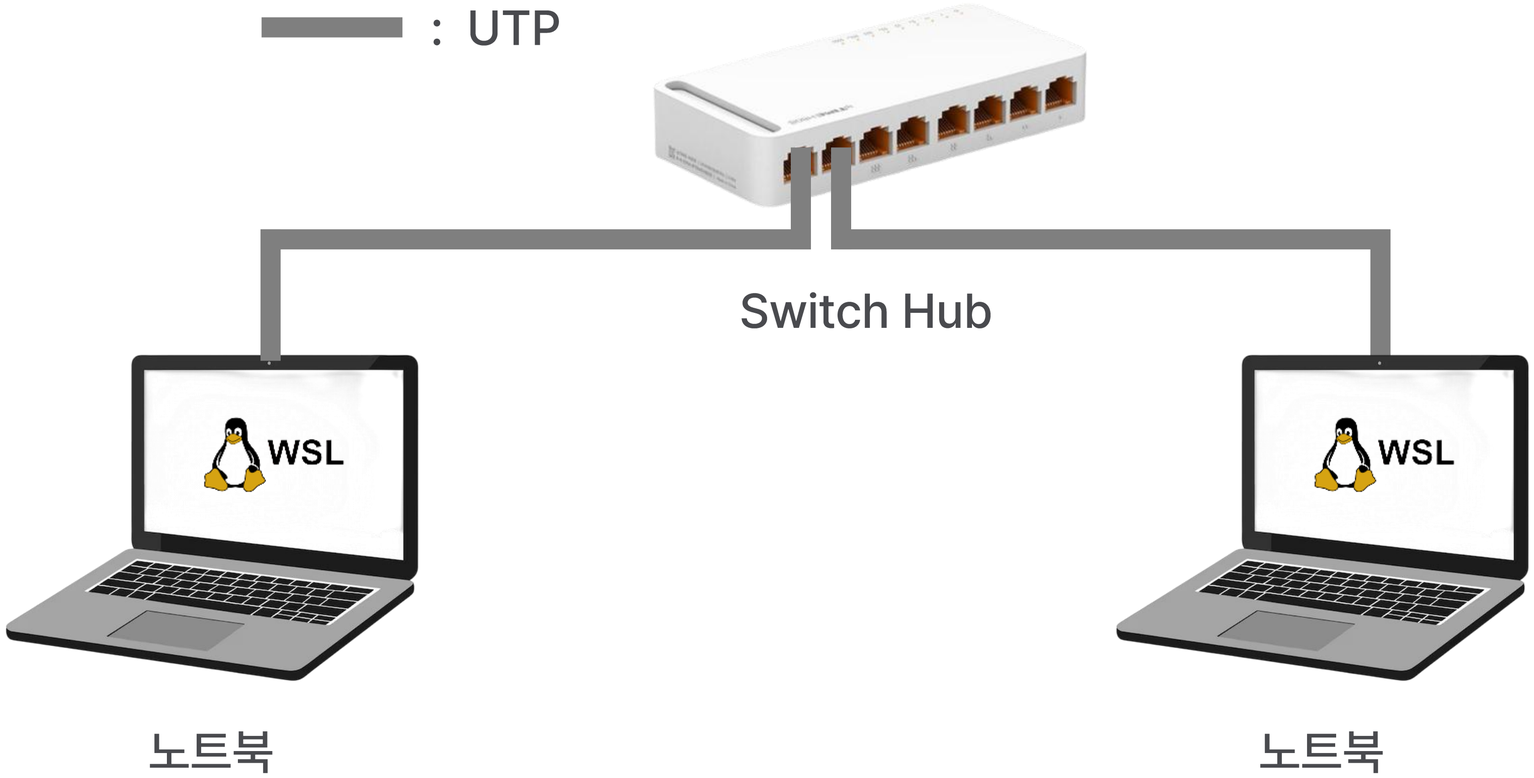
디지털 서명 및
무결성 검증 모듈 구현
(OpenSSL)

최재훈

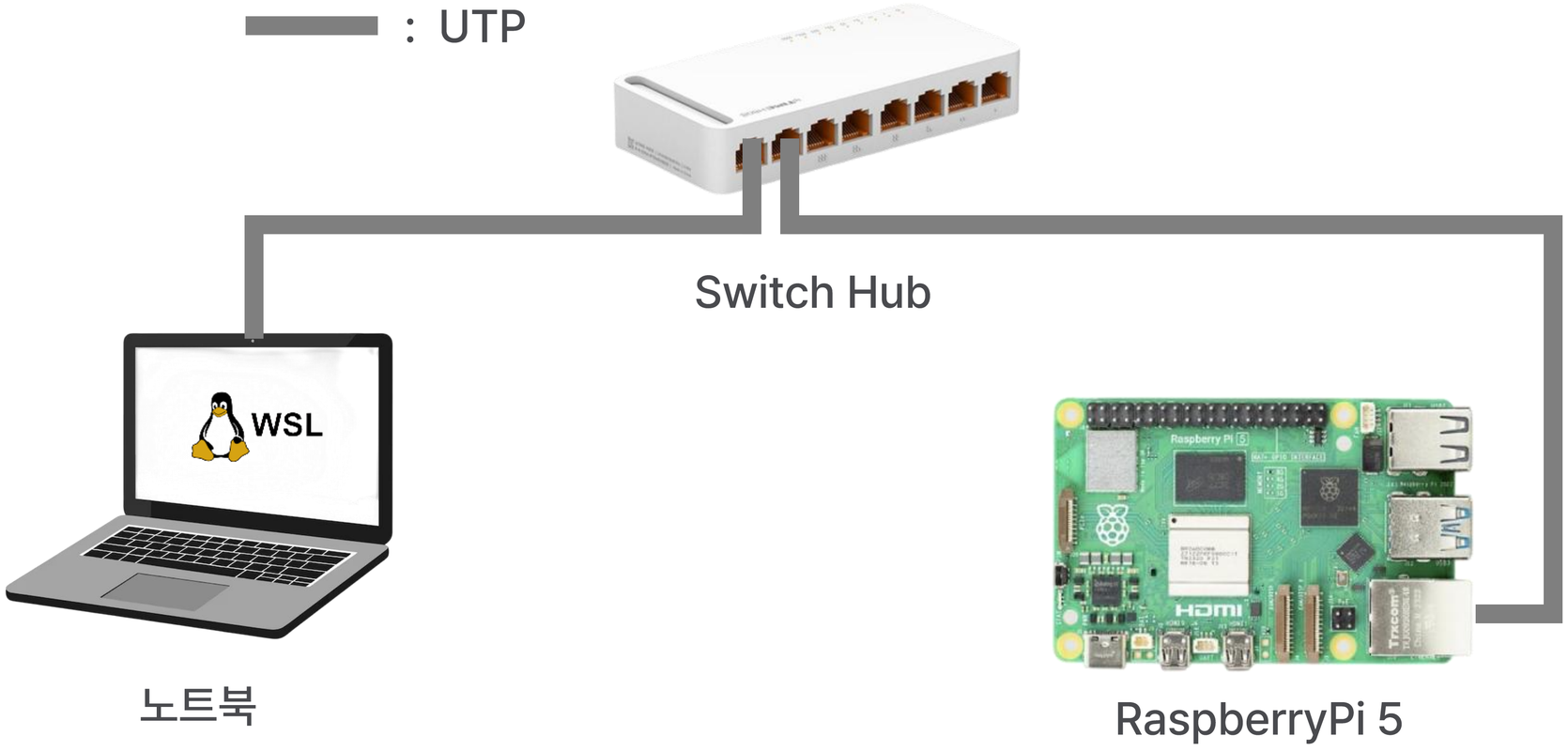


개발 환경 구성
기능 통합 및 테스트
보고서 작성

설계 사양(현재 테스트 환경)



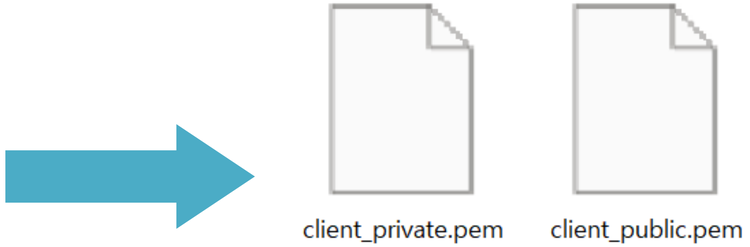
설계 사양(최종 적용 환경)



사전 준비 – 키 쌍 생성

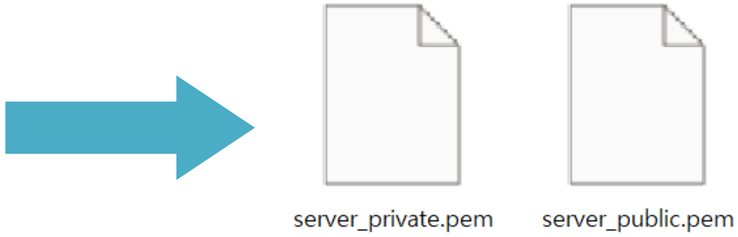
```
# 클라이언트 개인키 생성
openssl genpkey -algorithm ed25519 -out client_private.pem

# 클라이언트 공개키 추출
openssl pkey -in client_private.pem -pubout -out client_public.pem
```




```
# 서버 개인키 생성
openssl genpkey -algorithm ed25519 -out server_private.pem

# 서버 공개키 추출
openssl pkey -in server_private.pem -pubout -out server_public.pem
```



사전 준비 – 키 쌍 생성

클라이언트 측 보유

client_private.pem
(서명 생성 시 사용)

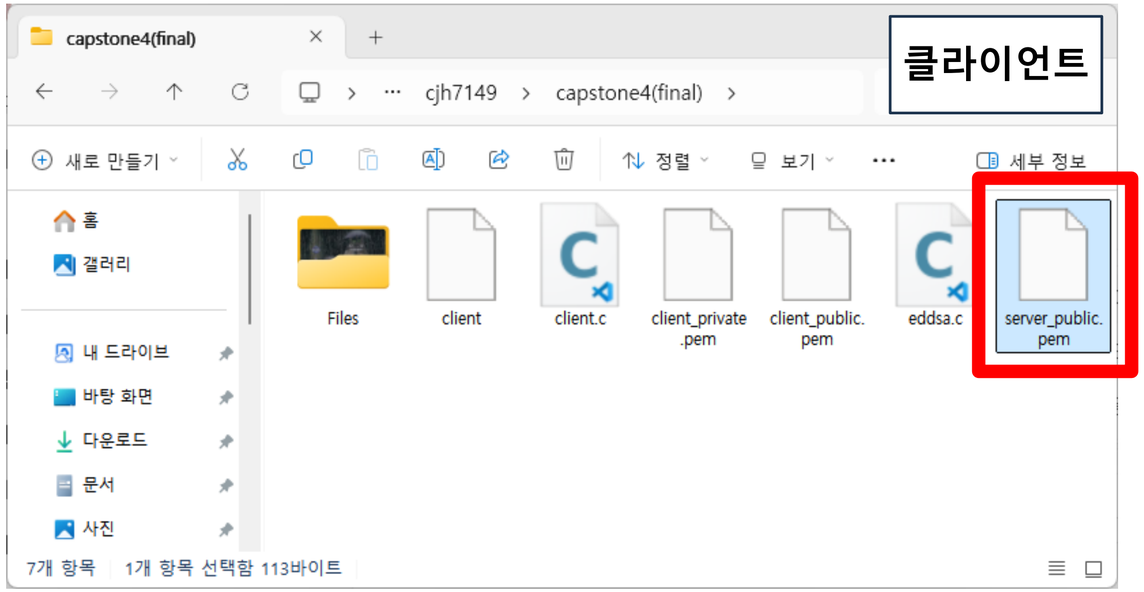
server_public.pem
(서명 검증 시 사용)

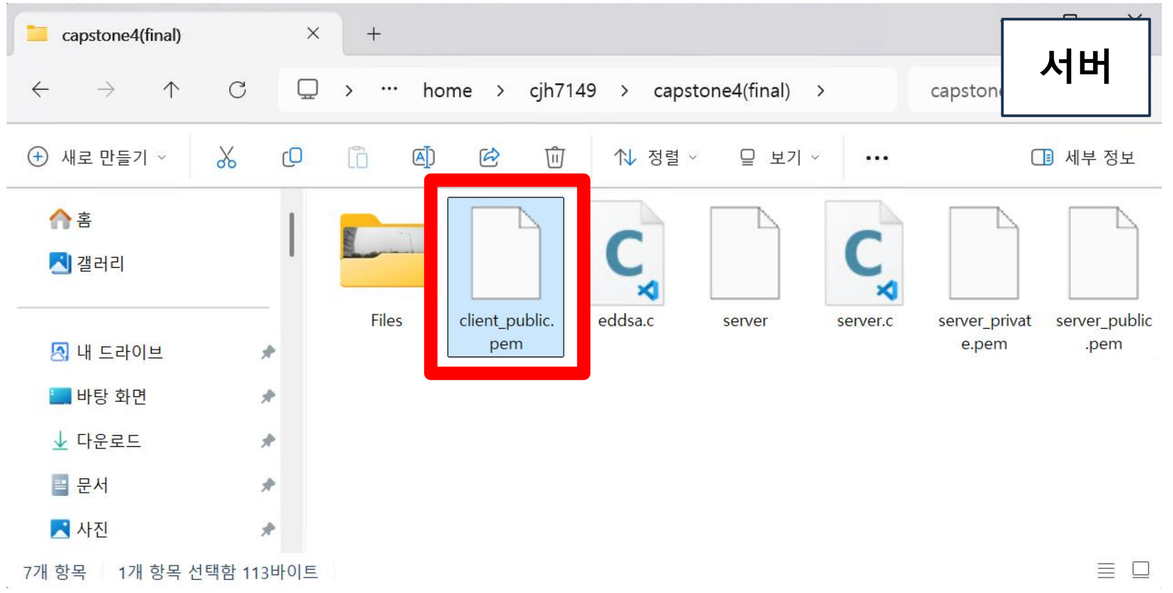
서버 측 보유

server_private.pem
(서명 생성 시 사용)

client_public.pem
(서명 검증 시 사용)

사전 준비 – 키 쌍 생성





각 주체는 자신의 개인키와 상대방의 공개키를 보관

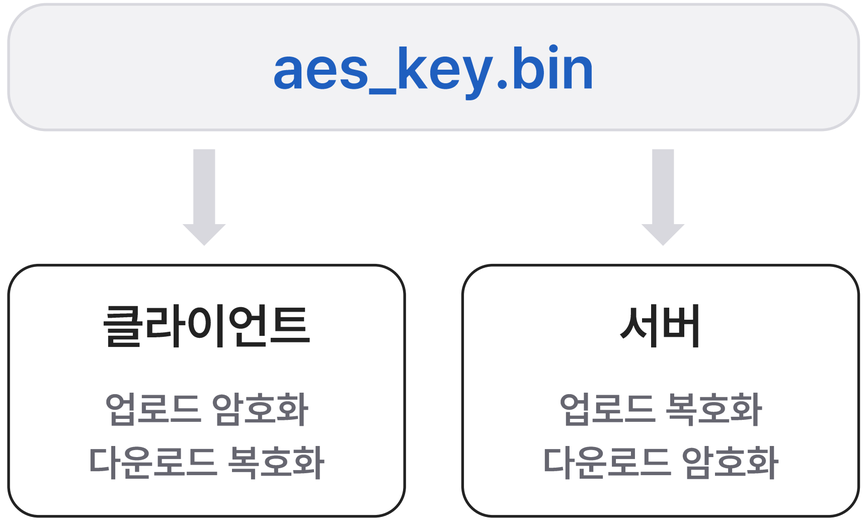
사전 준비 – AES 키 생성

```
# AES-256 대칭키 생성 (32 byte 난수)
openssl rand -out aes_key.bin 32

# 클라이언트 · 서버 양쪽에 동일한 파일 배치
```

aes.c

```
key_len = fread(key, 1, 32, fp);
ok = key_len == 32 && fgetc(fp) == EOF;
```



사전 공유 대칭키

시작 시 로드 · 종료 시 메모리에서 소거

서명 생성

client.c

```
// 파일 전체 + SHA-256 = 서명 입력
EVP_Digest(file, file_len, hash, ...);
memcpy(input + file_len, hash, 32);

eddsa_sign_with_key(input, len,
    &sign, &sign_len, CLIENT_PRIVATE_KEY);
```

eddsa.c

```
pkey = PEM_read_PrivateKey(fp, ...);

// 해시 알고리즘 NULL = EdDSA 내장
EVP_DigestSignInit(ctx, NULL, NULL, NULL, pkey);

// 64 byte 서명 생성
EVP_DigestSign(ctx, sig, &sig_len, msg, msg_len);
```

1. 파일 + SHA-256
서명 입력 구성

2. 개인키 로드
client_private.pem

3. EdDSA 초기화
해시 알고리즘 NULL

4. 서명 생성
64 byte

AES-256-GCM 암호화

aes.c

```
EVP_EncryptInit_ex(ctx, EVP_aes_256_gcm(), ...);
EVP_CIPHER_CTX_ctrl(ctx, SET_IVLEN, 12);
EVP_EncryptInit_ex(ctx, key, iv);

// AAD = 파일명 · 크기 · 해시
EVP_EncryptUpdate(ctx, NULL, aad, aad_len);
EVP_CIPHER_CTX_ctrl(ctx, GET_TAG, 16, tag);
```

client.c

```
aes_generate_iv(iv);       // 12 byte 난수
aes_gcm_encrypt_init(key, iv, aad);

while (남은 데이터) {      // 64 KB 단위
   aes_gcm_encrypt_update(plain → enc);
   send_all(enc);
}
send_all(tag, 16);         // 인증 태그
```

1. IV 생성
12 byte 난수

→

2. AAD 등록
파일명 · 크기 · 해시

→

3. 암호화 · 전송
64 KB 단위

→

4. 인증 태그 전송
16 byte

AES-256-GCM 복호화

aes.c

```
EVP_DecryptInit_ex(ctx, EVP_aes_256_gcm(), ...);
EVP_DecryptInit_ex(ctx, key, iv);  // 수신한 IV
EVP_DecryptUpdate(ctx, NULL, aad, aad_len);

// 수신한 태그와 비교 → 불일치 시 실패
EVP_CIPHER_CTX_ctrl(ctx, SET_TAG, 16, tag);
ok = EVP_DecryptFinal_ex(ctx, ...) == 1;
```

server.c

```
aes_gcm_decrypt_init(key, info.iv, aad);

while (남은 데이터) {      // 64 KB 단위
   recv_all(enc);
   aes_gcm_decrypt_update(enc → tmp);
}
recv_all(tag, 16);
ok = aes_gcm_decrypt_final(tag);
```

1. IV · AAD 설정
헤더에서 수신

→

2. 복호화
64 KB 단위 → tmp

→

3. 태그 검증
실패 시 즉시 중단

→

4. 서명 검증으로
성공 시에만 진행

서명 검증

수신 데이터
헤더 · 서명 · 암호문 · 태그

↓

AES-GCM 복호화 · 태그 검증
실패 시 중단

↓

검증 입력 재구성
복호화 파일 + 수신 해시

↓

Ed25519 서명 검증
상대방 공개키

↓

저장
검증 성공 시에만

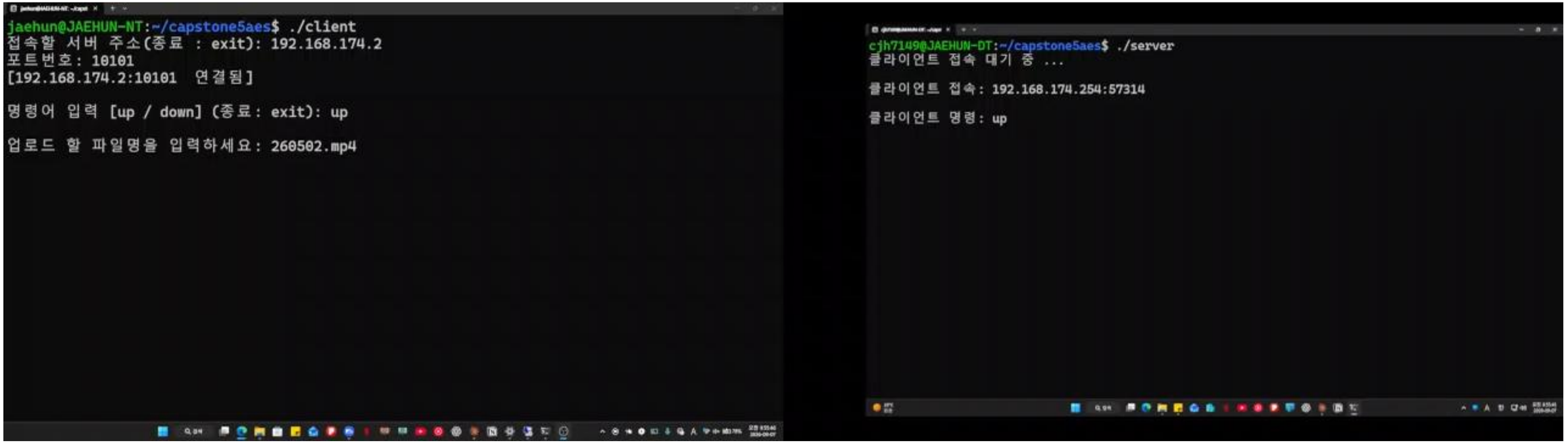
server.c

```
ok = recv_encrypted_file_data(...);

// AES 성공 시에만 서명 검증
sign_ok = ok &&
    eddsa_verify_with_key(input, len,
        sign, 64, CLIENT_PUBLIC_KEY);

save_ok = sign_ok && rename(tmp, save);
```

시연 영상 – 업로드 및 다운로드



연구 일정

연구내용	3월	4월	5월	6월	7월	8월	9월	10월
관련 내용 학습	■	■	■					
개발 환경 구축		■						
파일전송, 디지털 서명 모듈 구현		■	■	■				
AES 암호화 모듈 구현					■	■		
기능 개선							■	
라즈베리파이 동작 시험							■	
발표 자료 제작 및 시연 준비							■	■
최종 발표 및 산출물 제출								■

연구 일정

연구내용	3월	4월	5월	6월	7월	8월	9월	10월
관련 내용 학습	■	■	■					
개발 환경 구축		■						
파일전송, 디지털 서명 모듈 구현		■	■	■				
AES 암호화 모듈 구현					■	■		
기능 개선							■	
라즈베리파이 동작 시험							■	
발표 자료 제작 및 시연 준비							■	■
최종 발표 및 산출물 제출								■

프로젝트 향후 계획

대칭키 사전 공유 → X25519 키 교환 도입

라즈베리파이 환경에서 동작 확인

감사합니다

팀 해시브라운 